

Lecture: 1001

Von Neumann vs Harvard

How We Organise a Whole Computer

Two foundational blueprints for wiring a CPU to its memory — and the bottleneck that shapes them both.

CSA222: Modern Computer Architecture

We Have All the Pieces

Modules 1–2 gave us computation and memory. Now: how do they fit together?



Logic & arithmetic

Gates, adders, the ALU — the CPU can compute (Module 1).



Storage

Flip-flops, registers, register files, SRAM/DRAM (Module 2).



But no blueprint yet

How do CPU, memory, and I/O connect into a working machine?



Today: the organisation

Two classic answers — Von Neumann and Harvard.

Module 3 zooms out from circuits to the whole machine: organisation, then the instruction set it runs.

Today's Roadmap

Two blueprints, one famous bottleneck, and what real chips actually do.

1

Von Neumann

One memory for code AND data, one bus. Simple, universal — but bottlenecked.



2

Harvard

Separate instruction & data memories and buses. Faster, used in DSPs & MCUs.



3

Modified Harvard

What modern CPUs really do: unified DRAM, but split L1 caches.

The thread: every design is a response to one question — how fast can you move instructions and data between CPU and memory?

From Gates to a Complete Machine

A computer is more than an ALU — it needs memory, I/O, and a way to wire them together.



The question of computer organisation: how exactly do these connect — and how many buses?

The answer shapes everything: performance, cost, and how you program the machine. Two classic blueprints follow.

1. Von Neumann

One memory holds both instructions and data, reached by a single bus. The stored-program idea that started it all (1945).

The Stored-Program Concept

The radical 1945 idea: keep the program IN memory, as numbers, right alongside the data.

BEFORE vs AFTER

Before: computers were 'wired' for one task — reprogramming meant rewiring by hand.

After (von Neumann): the program is stored in the same memory as data, encoded as numbers.

*To run a different program, you just load different numbers.
Software as we know it was born.*

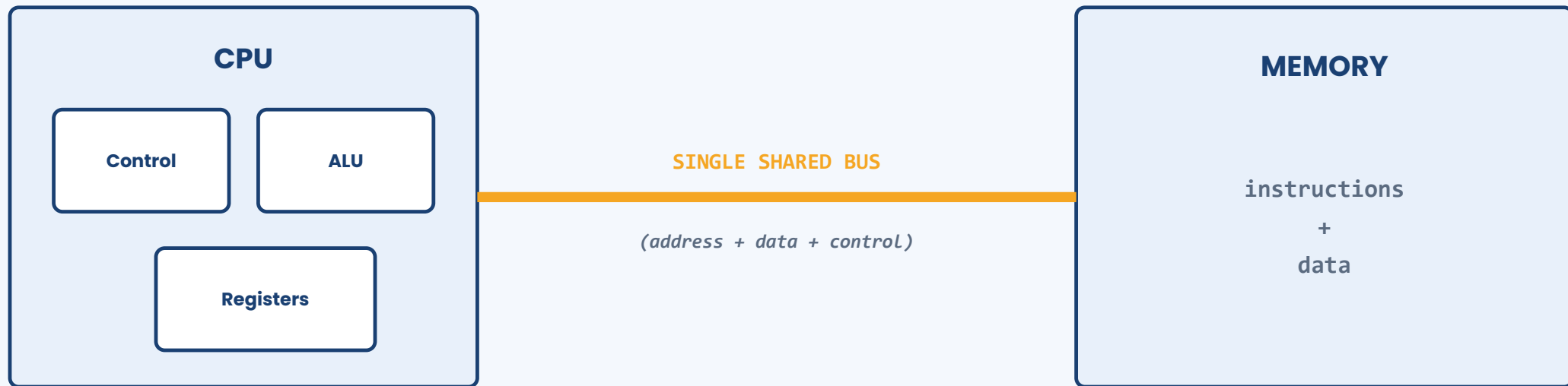
ONE MEMORY, TWO ROLES

0x00	lw r1, 0x40	instr
0x04	add r3, r1, r2	instr
0x08	sw r3, 0x44	instr
0x40	0x0000002A	data
0x44	0x00000000	data

Same memory, same address space — instructions and data live side by side.

Von Neumann Architecture

One memory, one bus — everything shares a single path between CPU and memory.



Instructions and data take turns on the ONE bus — the defining feature, and the built-in limit.

The Four Classic Components

Every von Neumann machine is built from these — the model still taught worldwide.



Control Unit

Fetches and decodes instructions, then directs everything else. The 'conductor' (an FSM, from L7).



ALU

The Arithmetic Logic Unit — does the actual computation: add, subtract, compare, AND/OR (Module 1).



Memory

One unified store holding both program instructions and data, addressed uniformly.



I/O

Input/output — keyboards, screens, disks; how the machine communicates with the world.

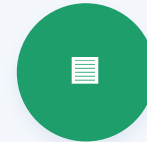
The Key Insight: Programs ARE Data

Because code lives in memory as numbers, it can be treated like any other data.



Compilers & assemblers

A program that writes another program — it outputs instructions as data into memory.



Loading & the OS

The OS loads a program by copying its instruction-bytes into memory, then jumps to them.



Just-in-time compilation

JITs (JVM, browsers) generate machine code at runtime and then execute it — data becomes code.



Self-modifying code

Programs can, in principle, rewrite their own instructions — powerful, and a security minefield.

This blurring of code and data is von Neumann's superpower — and the root of whole classes of security exploits.

A Bit of History

The idea that named an era — and the people who really built it.

1945, PRINCETON

John von Neumann described the stored-program design in the 'First Draft of a Report on the EDVAC' (1945).

The concept drew on work by Eckert, Mauchly, and Turing — von Neumann wrote it up, and the name stuck.

Nearly every general-purpose computer since — from the 1950s to your laptop — follows this basic model.

WHY IT WON

Simplicity: one memory, one bus — cheap and flexible.

Generality: the same hardware runs any program you load.

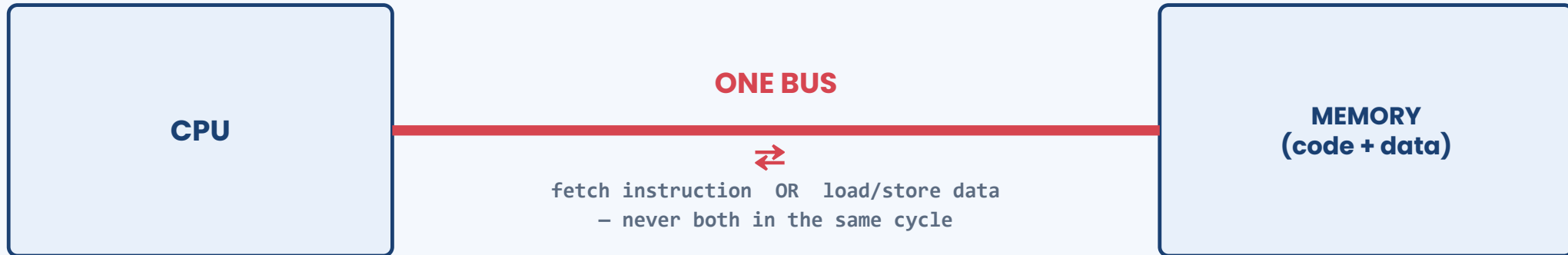
It was 'good enough' and easy to build — so it became the default, and the bottleneck it carries became the problem the next 70 years of architecture would attack.

2. The Von Neumann Bottleneck

One bus, two jobs: instructions and data can't both cross at once. This single limit shapes modern architecture.

The Von Neumann Bottleneck

Every instruction fetch and every data access share ONE bus — so they must take turns.



The CPU can compute far faster than the single bus can feed it — so the bus becomes the limit.

This traffic jam on the memory bus is 'the Von Neumann bottleneck' — the term coined by John Backus in 1977.

Why It Limits Performance

A fast CPU stalls whenever the bus is busy — computation waits on communication.



Bus contention

Instruction fetch and data access compete for the same wires. One must wait for the other.



Idle compute units

While the ALU waits for the bus, it does nothing. Raw compute power sits unused.



Caps achievable IPC

Instructions-per-cycle is limited not by the ALU but by how fast the bus delivers work.



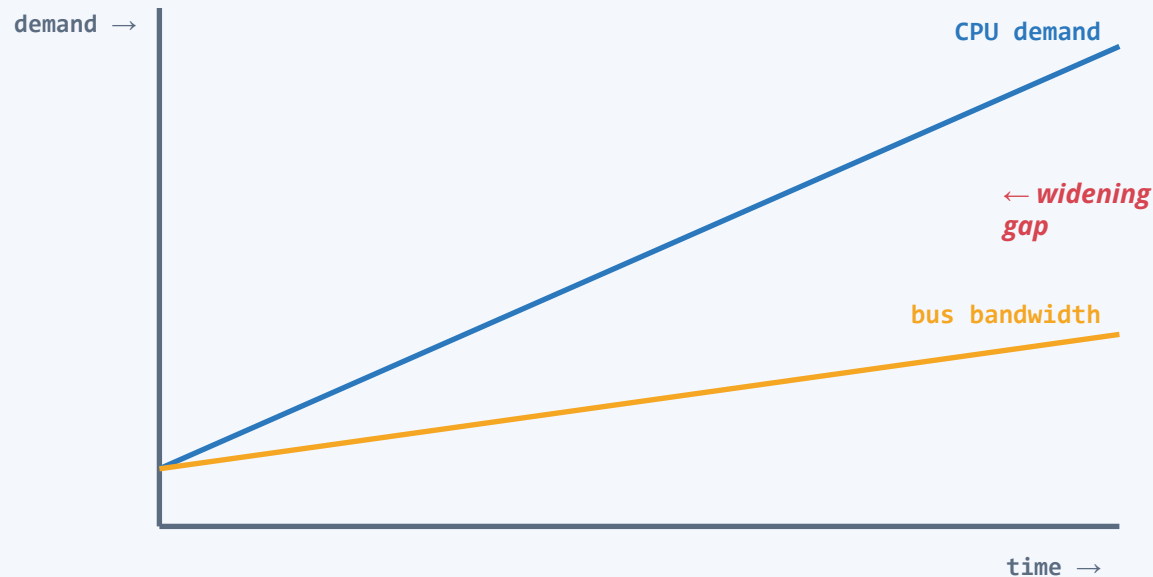
Worsens with CPU speed

Faster CPUs need data faster — but the bus doesn't scale as quickly. The gap grows.

The bottleneck is why caches, wider buses, and Harvard-style splits exist — all attempts to widen this pinch.

Why It Gets Worse Over Time

CPU speed raced ahead of memory bandwidth for decades — the bottleneck tightened.



A faster CPU just spends more cycles waiting on the same bus.

THE RESPONSE

Architects fought back with:

- Caches — keep hot code & data on-chip (Module 5).
- Wider & faster buses — move more bits per cycle.
- Harvard-style splits — separate paths for code and data.

The rest of today — and much of this course — is about that last idea.

Quick Check

Show of hands — no notes.

Q. The Von Neumann bottleneck exists because...

A

The ALU is too slow

B

One bus is shared by code & data

C

Memory is too small

D

There are too many registers

✓ *they take turns on one path*

The single shared bus means an instruction fetch and a data access can't happen simultaneously — that's the bottleneck.

3. Harvard Architecture

Separate memories and buses for instructions and data — so the CPU can fetch and load at the same time.

Harvard Architecture

Two memories, two buses — instructions and data never compete for the same path.



Instruction fetch (I-bus) and data access (D-bus) happen in the SAME cycle — no contention.

Two Buses, Twice the Throughput

The whole point: parallel access. What took two cycles now takes one.

VON NEUMANN

one bus, take turns

- Cycle 1: fetch instruction.
- Cycle 2: access data.
- Two cycles for fetch + data.
- Bus is the bottleneck.

HARVARD

two buses, in parallel

- Cycle 1: fetch instruction AND access data.
- Both buses active at once.
- One cycle for fetch + data.
- Up to 2× memory throughput.

The cost: two memories and two bus systems — more wires, more pins, more silicon. Nothing is free.

Where the Name Comes From

The Harvard Mark I (1944) kept instructions on punched tape, data in relays — physically separate.

THE HARVARD MARK I

Built at Harvard in 1944, it stored instructions on punched paper tape and data in electro-mechanical relay counters.

Because instructions and data lived in physically different places, the machine could read the next instruction while working on data.

That separation gave the architecture its name — even though today it means split buses, not tape and relays.

THEN vs NOW

1944: instructions on tape, data in relays — separate by physical necessity.

Today: separate on-chip memories/caches with separate buses — separate by design choice.

Same core idea across 80 years: give code and data their own path, and they stop fighting.

One Cycle, Two Transfers

A timeline makes the advantage obvious — Harvard overlaps what Von Neumann serialises.



Same work, half the time — because the two transfers ride separate buses simultaneously.

Where Harvard Shines

When predictable, high-throughput access matters more than flexibility.



Digital Signal Processors

DSPs stream data through fixed algorithms (filters, FFTs). Parallel code+data fetch keeps the pipeline fed.



Microcontrollers

The ATmega328 (Arduino Uno) is true Harvard — flash for code, SRAM for data, separate buses.



Real-time systems

Deterministic timing matters. No bus contention means predictable, repeatable cycle counts.



Embedded & IoT

Small, fixed programs in flash + small data in RAM map naturally onto the Harvard split.

Von Neumann vs Harvard

The trade-off in one table: simplicity & flexibility vs throughput & predictability.

	VON NEUMANN	HARVARD
Memory	One (unified)	Two (separate I & D)
Buses	One shared	Two independent
Fetch + data	Sequential (2 cycles)	Simultaneous (1 cycle)
Throughput	Bottlenecked	Up to 2× higher
Cost / complexity	Lower	Higher (more wires/pins)
Flexibility	High (code = data)	Lower (separate spaces)
Typical use	General-purpose CPUs	DSPs, microcontrollers

Neither is 'better' — general CPUs pick flexibility, embedded/DSP picks throughput. Modern chips blend both...

4. Modified Harvard

What your laptop actually is: Von Neumann at main memory, Harvard inside the chip. The best of both.

The Real World: Modified Harvard

Modern CPUs are Von Neumann on the outside, Harvard on the inside.

TWO LEVELS, TWO ANSWERS

Outside the chip: one unified DRAM holds code and data — pure Von Neumann. Simple, flexible, cheap.

Inside the chip: the L1 cache is SPLIT into a separate instruction cache (L1-I) and data cache (L1-D) — Harvard-style.

So the CPU fetches code and data in parallel from L1, but everything still lives in one memory underneath.

BEST OF BOTH

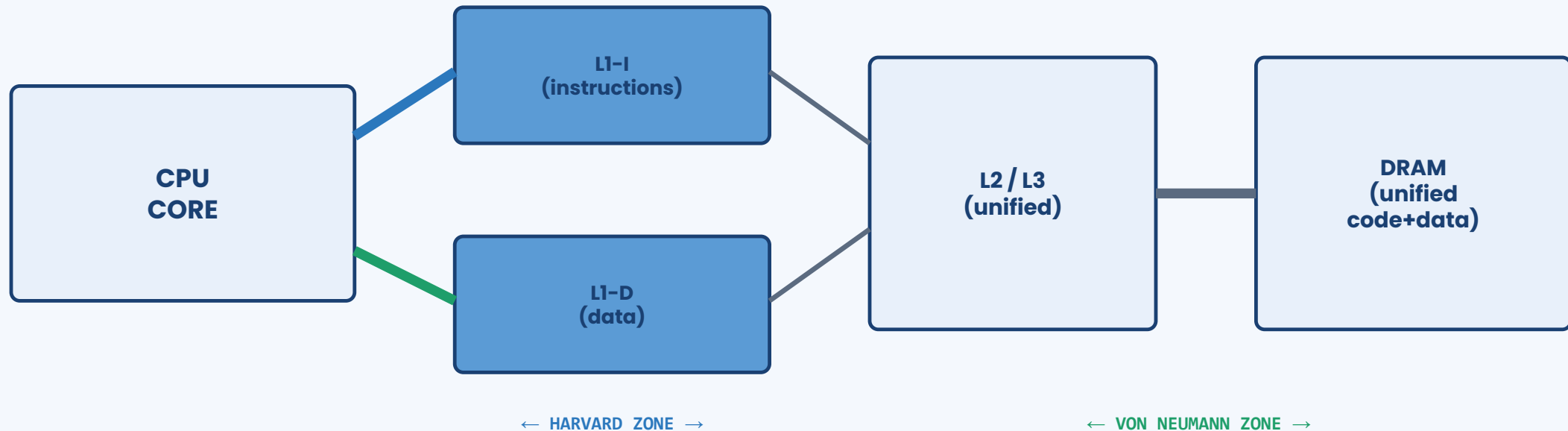
Flexibility of Von Neumann:
one address space, code = data, easy to program & load.

Speed of Harvard:
split L1 caches feed the pipeline code + data every cycle, no contention.

This hybrid is why the split L1 exists — a direct preview of Module 5.

Split L1-I and L1-D Caches

Where the two worlds meet: unified memory below, split caches above.



Split at L1 for parallel fetch; unified from L2 down for simplicity. Modified Harvard in one picture.

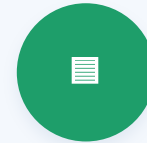
Why This Hybrid Wins

It captures each architecture's strength exactly where that strength matters most.



Parallel L1 access

Split L1-I/L1-D lets the pipeline fetch an instruction and a data word every cycle — Harvard speed where it counts.



Unified main memory

One DRAM address space keeps loading, linking, and programming simple — Von Neumann flexibility.



Cache coherence handled

Hardware keeps the split caches consistent with unified memory, so software sees one memory.



Scales to real workloads

General-purpose speed AND generality — which is why essentially every modern CPU does this.

Spot the Architecture (Activity 3)

Match each real system to its nearest architectural model.

ATmega328 (Arduino Uno)

True Harvard

Flash for code, SRAM for data, separate buses — textbook Harvard.

ARM Cortex-M4

Modified Harvard

Separate I/D buses on-chip (Harvard-ish) over a unified address space.

x86-64 desktop (Intel/AMD)

Modified Harvard

Unified DRAM (Von Neumann) with split L1-I / L1-D caches inside.

Rule of thumb: small embedded = true Harvard; anything with caches = modified Harvard; pure Von Neumann is now rare.

Worked Example: Peak Bandwidth

The Activity-1 calculation, fully worked — the numbers behind the bottleneck.

GIVEN: 1 GHz CPU • fetches one 32-bit instruction + one 32-bit data word every cycle

- | | | |
|---|--------------------|---|
| 1 | Per cycle | 32 bits (instr) + 32 bits (data) = 64 bits total must move |
| 2 | Per second | 64 bits × 1×10^9 cycles/s = 64 Gbit/s = 8 GB/s total |
| 3 | Von Neumann | ONE bus must carry all 8 GB/s → it saturates first (the bottleneck) |
| 4 | Harvard | I-bus 4 GB/s + D-bus 4 GB/s, in parallel → no single-bus saturation |

Takeaway: same 8 GB/s of work, but Harvard splits it across two parallel buses so neither becomes the single choke point. That's the whole argument for splitting.

Common Mistakes to Avoid

The slips that cost marks — and reveal shaky understanding.



'Harvard has no bottleneck'

It eases the fetch-vs-data one, but memory bandwidth limits still exist — it's not magic.



Forgetting 'programs are data'

That's the stored-program idea — the heart of Von Neumann, not a footnote.



Confusing the two names

Von Neumann = ONE memory/bus. Harvard = TWO. Don't swap them under exam pressure.



Ignoring cost of Harvard

Two buses/memories mean more pins, wires, and area. Throughput isn't free.



'Modern PCs are pure Harvard'

They're MODIFIED Harvard: unified DRAM, split L1 caches. Both models at once.



Bus width $\neq$ address space

Splitting buses (Harvard) is separate from how big memory is. Don't conflate them.

Key Takeaways

Five sentences — the organisation of every computer.

- 01 Von Neumann uses **ONE** memory and **ONE** bus for both instructions and data — the stored-program model (1945).
- 02 The Von Neumann bottleneck: instructions and data share one bus, so they take turns — capping performance.
- 03 Harvard uses **SEPARATE** instruction and data memories/buses, enabling simultaneous fetch and data access.
- 04 Harvard suits DSPs, microcontrollers, and real-time systems, where throughput and predictability matter most.
- 05 Modern CPUs are **MODIFIED** Harvard: unified DRAM (Von Neumann) with split L1-I / L1-D caches (Harvard) — the best of both.

Practice & Homework

Diagram, calculate, and classify.

01

Diagram Both

Draw Von Neumann and Harvard block diagrams side by side. Label CPU, memories, and every bus.

02

Bandwidth

A 2 GHz CPU fetches a 64-bit instruction + 64-bit data per cycle. Find the bus bandwidth for each architecture.

03

Classify

For an ATmega328, ARM Cortex-M4, and x86-64 desktop, name the nearest architecture and justify it.

EXIT TICKET

1. In one sentence, state the Von Neumann bottleneck.
2. Why can a Harvard machine fetch an instruction and a data word in the same cycle?
3. How does an x86 chip's split L1 cache reflect Harvard thinking?

Rapid-Fire Recap

Three questions — shout the answers.

Q1

Von Neumann uses how many buses?

one

Q2

Harvard separates instructions from...

data

Q3

Modern CPUs split which cache?

L1

All three instant? You understand how a computer is organised — next we look at its instruction set.

END OF L09

One memory or two? Now you decide.

NEXT — L10

The Instruction Set Architecture — the CPU's vocabulary: what an instruction is, how it's encoded, and RISC vs CISC.